

3DENGINE

Complete Systems Reference

A source-aligned guide to the engine, editor, native asset pipeline, runtime, and gameplay framework.

JULY 2026 EDITION

Includes level-as-asset streaming, C++ and Lua scripting, named timers, global time dilation, hit stops, animation, physics, AI, particles, audio, HUD, cameras, shaders, native assets, cooking, and diagnostics.

109 runtime headers | 35 editor headers | 14 system chapters | source coverage appendix

Contents

- [01 Systems overview](#)
- [02 Core application and project architecture](#)
- [03 ECS, scenes, worlds, and level streaming](#)
- [04 Native assets, importing, loading, and cooking](#)
- [05 Rendering, lighting, cameras, environment, terrain, and water](#)
- [06 Materials, textures, and shader graphs](#)
- [07 Animation, graphs, blend spaces, actions, sockets, and characters](#)
- [08 Physics, collision, controllers, joints, and ragdolls](#)
- [09 AI, navigation, perception, behavior trees, and blackboards](#)
- [10 Gameplay framework, C++ and Lua scripting, timers, and global time](#)
- [11 Audio engine, sources, cues, mixer, music, and editor](#)
- [12 Particle simulation, modules, rendering, and scripting](#)
- [13 Runtime UI, HUD documents, bindings, fonts, and editor](#)
- [14 Editor application, panels, asset editors, and Play mode](#)
- [15 Build, standalone player, packaging, diagnostics, and tests](#)
- [16 Public source coverage appendix](#)

01 Systems overview

This reference documents the systems currently implemented in the engine, editor, game, and player targets. It is based on the public headers and their current implementations, not on the older roadmap.

The engine is C++17, uses OpenGL, and follows one central boundary:

- engine is the reusable runtime library.
- game contains project gameplay scripts and may depend on engine.
- editor authors assets and scenes and may depend on both.
- player loads exported runtime scenes without depending on editor code.

Read by subject

- 1 Core application and project architecture
- 2 ECS, components, systems, prefabs, scenes, worlds, and level streaming
- 3 Native assets, registry, importing, loading, and cooking
- 4 Rendering, lighting, cameras, environment, terrain, and water
- 5 Materials, textures, and shader graphs
- 6 Skeletal animation, graphs, blend spaces, actions, sockets, and characters
- 7 Physics, collision filtering, queries, joints, controllers, and ragdolls
- 8 AI, navigation, perception, behavior trees, blackboards, and BT scripts
- 9 Gameplay framework, player controller, camera direction, and scripts
- 10 Audio engine, sources, cues, mixer, music, and editor
- 11 Particle simulation, modules, effects, rendering, and scripting
- 12 Runtime UI, HUD documents, bindings, fonts, and HUD editor
- 13 Editor application, panels, controllers, asset editors, and Play mode
- 14 Build, standalone player, packaging, diagnostics, and tests
- 15 Public source coverage appendix

Runtime frame flow

```
Window events
-> variable Update
    unscaled clock, time dilation, input, scripts, audio, animation, camera, UI
-> zero or more fixed updates
    game mode, scripts, gameplay, AI, controllers, ragdoll activation,
    physics, projectile/contact processing
-> Render
    shadows, PBR/static/skinned geometry, environment, particles,
    post process, HUD, debug/editor overlays
-> swap buffers
```

Physics and gameplay that affect collision should run at the fixed step. Presentation systems such as animation blending, audio listener updates, camera effects, and UI normally run once per rendered frame.

Gameplay timers, scripts, animation, AI, particles, cameras, and physics consume the globally scaled gameplay delta. Editor UI and audio mixing remain on unscaled time. Hit-stop duration is also counted with unscaled time, allowing a complete gameplay freeze to release reliably.

Asset and scene flow

```
Source file (FBX/OBJ/gltf/PNG/JPEG)
-> editor import settings
-> engine-owned native asset + stable AssetHandle
-> AssetRegistry.3dgdb
-> authored scene/character/material/graph references ID + fallback path
-> RuntimeAssetManager resolves and caches GPU/runtime objects
-> AssetCooker copies only dependencies reachable from the runtime scene
```

Documentation conventions

- "Authored asset" means an editable document such as .3dgmat, .3dgshader, .particle, .hud, .3dgcharacter, .3dgprefab, .3dgclip, .3dggraph, or .btgraph.
- "Native asset" means an engine-owned binary such as .3dgmesh, .3dgskmesh, .3dgskel, .3dganim, or .3dgtex.
- "Runtime scene" means the exported scene consumed by Editor Play and the standalone player.
- Paths shown under Content are project-relative in authored data. Stable asset IDs are authoritative after import; paths remain readable fallbacks.

Related task-oriented guides

- First game tutorial
- Fireball spell tutorial
- Audio system guide
- Existing engine manual

02 Core application and project architecture

Targets and dependency direction

The root CMake project builds these primary targets:

Target	Role
engine	Reusable static runtime library
game	Shared project gameplay module and script registration
3DGEEditor	Dear ImGui authoring application
3DGScriptCompiler	Helper used by editor-managed script compilation
player	Standalone packaged-game runtime
tests/*	Regression executables grouped by subsystem

Runtime code must not include editor headers. Editor documents are converted into engine runtime descriptions or exported scene records before the player loads them.

Third-party foundations are GLFW, GLAD, GLM, OpenGL, Assimp, Dear ImGui, and miniaudio. PNG, JPEG, and TrueType handling also have engine-side implementations.

Application

`engine::Application` owns a `Window` and defines the main-loop contract:

- `OnInit()` runs after the OpenGL context exists.
- `OnUpdate(dt)` runs once per rendered frame.
- `OnFixedUpdate(fixedDt)` runs zero or more times from an accumulator.
- `OnRender()` draws one frame.
- `OnShutdown()` releases application-owned resources.

The default fixed step is 1/120 second. `InterpolationAlpha()` exposes the remaining accumulator fraction for presentation interpolation. Change the simulation rate with `SetFixedTimeStep()` before calling `Run()`.

Do not perform OpenGL resource creation in constructors that run before `OnInit()`. Editor panels follow the same rule by creating preview resources lazily during `Draw()`.

Window

`engine::Window` is the RAII wrapper for GLFW and the OpenGL context. It provides:

- framebuffer size and aspect ratio;
- key and mouse-button state;
- absolute mouse position, per-frame mouse delta, and wheel delta;
- cursor capture for gameplay mouse look;
- fullscreen and VSync control;
- minimized-state detection;
- access to the native `GLFWwindow`.

`Window::Update()` swaps buffers and pumps OS events. Mouse deltas are reset per frame, so consumers should read them during the application update.

Configuration and paths

`engine::Config` reads and writes simple `key = value` files. Typed getters always accept a fallback, so an absent setting is not fatal. The standalone player uses this for window and startup configuration.

`engine::ExecutableDir()` returns the executable directory and is the preferred base for packaged content. It prevents runtime behavior from depending on the process working directory.

`HighPerformanceGPU.h` exports the NVIDIA Optimus and AMD PowerXpress hints on Windows. Include it exactly once in an executable entry point.

Timing ownership

Use the two clocks consistently:

Work	Recommended clock
Keyboard/mouse sampling	Variable update
Camera input and camera effects	Variable update
Animation visual blending	Variable update
Audio source/listener updates	Variable update
Physics and character collision	Fixed update
AI steering and path following	Fixed update
Projectile sweeps and damage	Fixed update
Script <code>OnUpdate</code>	Variable update
Script <code>OnFixedUpdate</code>	Fixed update

Avoid applying the same movement from both clocks. Doing so makes speed frame-rate dependent and can cause collision tunnelling or jitter.

Ownership rules

GPU wrappers such as `Mesh`, `Texture`, `Shader`, `Framebuffer`, and `Cubemap` own OpenGL objects and are move-only. Runtime asset caches own shared models and textures; ECS components store non-owning pointers to those cached resources. Clear or replace a registry before destroying the cache that its components reference.

The hot-reloadable `ScriptModule` has a stricter lifetime rule: destroy all live script instances and clear the script factory registry before unloading the DLL, because their code and virtual tables live inside that module.

03 ECS, scenes, worlds, and level streaming

Registry model

`engine::ecs::Registry` stores integer Entity identifiers and type-specific component pools. The important operations are:

- `Create()` and `Destroy(entity);`
- `Add<T>`, `Remove<T>`, `Has<T>`, `Get<T>`, and `TryGet<T>`;
- multi-component `view<T...>()` iteration.

Components are data. Behavior lives in systems, controllers, or scripts. This keeps runtime data serializable and lets the editor clone a scene registry for Play mode.

Core scene components

Component	Purpose
Transform	Position, quaternion rotation, scale, and model matrix
RuntimeName	Stable authored object name used for lookups and bindings
MeshRenderer	Primitive mesh pointer plus flat tint
MeshPBR	Primitive mesh plus full PBR material
ModelAsset / SkinnedModelAsset	Unresolved authored asset references
LoadedModelAsset	Resolved shared static model
AnimatedModel	Resolved skeletal model and per-entity animation state
MaterialAsset	Authored material path and instance overrides
LoadedMaterialAsset	Resolved material maps and optional shaders
Light	Directional, point, spot, or area light data
LinearVelocity / AngularVelocity	Simple transform motion
Rotator / Mover	Reusable authored gameplay motion
AudioSource	Serializable audio-source settings
TriggerAudioAction	Collision-driven no-code audio action

Physics, gameplay, AI, particles, and scripts add their own components covered in their respective chapters.

Built-in ECS systems

`ecs/Systems.h` contains rendering bridges:

- `RenderMeshes` draws `Transform` + `MeshRenderer`.
- `RenderLoadedModels` draws imported `LoadedModelAsset` objects, selects a custom material shader when present, uploads instance parameters, and applies complete model-material overrides.
- `LoadedModelMaterialOverride` converts PBR material values to the legacy imported-model material interface.

`ecs/RuntimeSystems.h` contains small stateless updates:

- `UpdateRuntimeMotion` applies linear and angular velocity.
- `UpdateGameplay` updates `Mover` and `Rotator`, using rigid-body velocity for

dynamic or kinematic objects where appropriate.

Gameplay-specific systems such as health, projectiles, attachments, scripts, and ragdolls are documented separately.

Editor scene

EditorScene is the authoring representation. Each object contains:

- an ECS entity;
- its primitive or imported model reference;
- material and material-instance overrides;
- model-only orientation, position, and scale offsets;
- animation, physics, audio, particle, gameplay, script, AI, terrain, camera,

and prefab authoring state.

World transforms belong to the scene instance. Render-only model offsets fix asset orientation or visual alignment without rotating the collider, controller, navigation direction, or world axes.

The editor records undoable changes, selection, locks, visibility, component add/remove operations, and scene serialization. A locked object rejects mutating Inspector and editor commands.

Scene serialization and loading

The data flow is:

- 1 EditorScene saves the editable scene.
- 2 RuntimeSceneExporter writes a runtime-compatible scene.
- 3 RuntimeSceneLoader::Load parses it into a RuntimeSceneLoader::Scene.
- 4 RuntimeSceneLoader::Instantiate creates ECS entities and serializable components using the supplied primitive meshes.
- 1 RuntimeAssetManager::ResolveRegistryAssets resolves model, skeletal, texture, material, and shader references.
- 1 Host-owned systems build transient state such as live audio voices, particle emitters, behavior trees, controllers, and physics caches.

The scene description contains environment settings, game-mode settings, navigation bounds and agents, trigger actions, camera zones, joints, terrain, camera presets and sequences, lights, and entity records.

Stable asset IDs are stored with readable fallback paths. On load, the ID is resolved first so renamed or moved assets continue to work.

Worlds and level-as-asset streaming

A .3dgworld manifest treats a level as an engine-owned scene asset. A world contains one always-resident persistent scene and any number of placed streamed levels. Each LevelRef stores:

- a stable scene asset ID and readable fallback path;
- a placement transform applied to the entire level;
- local bounds used for streaming distance tests;
- load and unload radii;
- a Distance, AlwaysLoaded, or Manual streaming rule.

`LevelStreamingManager` instantiates active levels into the shared ECS registry and records exactly which entities each level owns. Unloading destroys that entity group while leaving shared GPU assets cached. Distance streaming uses separate load and unload radii to prevent rapid boundary thrashing, and the manager performs at most one activation or deactivation per update to reduce frame spikes.

Placed level transforms affect entity transforms, mover origins and axes, rotator and light directions, navigation bounds and patrol points, joint anchors, and camera presets. The persistent level owns world environment and game-mode state; streamed levels cannot replace it when they activate.

Activation hooks let the editor or player create host-owned state for newly loaded entities, such as physics bodies, audio voices, animation runtime data, navigation data, particle emitters, and scripts. The before-deactivate hook must release that transient state before the entities are destroyed.

Manual streaming can be connected to doors, transitions, or script logic with `LoadLevel` and `UnloadLevel`. `UnloadAll` is used before replacing a world or shutting down the runtime.

Prefabs

There are two prefab concepts:

- `engine::ecs::PrefabLibrary` is the low-level runtime ECS prefab facility.
- The editor `.3dgprefab` asset captures an `EditorScene::Object` component

configuration for reusable authored objects.

The Prefab Editor workflow is:

- 1 Configure an object in the scene.
- 2 Open **Prefab Editor**.
- 3 **Capture from Selected**.
- 4 Save it under `Content/Prefabs`.
- 5 Add linked instances with **Add to Scene**.

A prefab captures model, material, render-only model offsets, physics, movement, health, and script configuration. Applying a prefab intentionally does not overwrite the instance's world transform or name. Saving the prefab re-applies its component configuration to linked scene instances.

Use a Character Asset instead of a generic prefab when the object requires a skeleton, animation graph, sockets, attachments, player settings, or AI setup.

Empty objects and system hosts

An Empty object has a transform and may carry scripts or other non-rendering components. It is appropriate for score managers, game-state coordinators, spawn managers, audio controllers, and other world systems. Its visibility toggle is treated as "Active in Play," and it does not create visible geometry.

Common ordering constraints

- Instantiate components before resolving assets.
- Apply a streamed level's placement before building host-owned physics, navigation, animation, or audio state.
- Resolve skeletal assets before constructing animation poses or sockets.
- Run health bookkeeping before checking `justDied`.
- Activate ragdolls before physics and synchronize ragdoll poses after physics.
- Process collision events only after `PhysicsWorld::Step`.

- Shut down scripts and audio voices before replacing the runtime registry.
- Release per-level transient systems before destroying a streamed level's entities.

04 Native assets, importing, loading, and cooking

Asset identity

AssetHandle is a stable 128-bit identifier. It is embedded in engine-owned assets and authored metadata and remains unchanged when an asset is renamed or moved.

AssetType currently identifies static meshes, skeletal meshes, skeletons, animations, materials, textures, audio, shaders, particles, particle effects, HUDs, characters, animation clips, animation graphs, behavior trees, scenes, scripts, terrain, and fonts.

An AssetReference stores:

- the authoritative AssetHandle;
- a readable fallback path for legacy projects and diagnostics.

ResolveAssetReference resolves by ID first, validates the expected type, then falls back to the path.

Native container

All binary engine-owned files begin with NativeAssetHeader, containing:

- container and asset versions;
- asset type and stable ID;
- importer version and source hash;
- payload size and flags;
- dependency IDs.

Current native extensions:

Extension	Asset
.3dgmesh	Static mesh
.3dgskmesh	Skeletal mesh
.3dgskel	Skeleton
.3dganim	Animation clips
.3dgtex	Texture

Raw FBX, OBJ, glTF, Collada, PLY, STL, PNG, and JPEG files are import sources, not the preferred runtime representation.

Asset registry

AssetRegistry maps IDs and normalized virtual paths to AssetRegistryEntry records. Each entry stores the type, virtual path, source file, source hash, importer version, and dependency IDs.

Principal operations:

- Register, Remove, and Move;
- Find, FindByPath, ByType, and Referencers;
- Validate for duplicate, missing, and type/dependency issues;
- Save and Load;
- RebuildFromContent;
- SynchronizeAuthoredAssets.

The project registry is `Content/AssetRegistry.3dgdb`. Editor startup synchronizes both native and authored assets before validation. This repairs registries after assets are copied or moved within Content and prevents valid native texture dependencies from being reported as missing.

Static-mesh import

`ImportStaticMeshToAsset`:

- 1 Uses Assimp to load a supported source.
- 2 Rejects skeletal sources so bone data is not silently discarded.
- 3 Applies scale, normals, tangent, vertex-join, and UV options.
- 4 Builds submeshes with position, normal, UV, and tangent data.
- 5 Embeds imported material data and RGBA textures.
- 6 Computes bounds, source hash, and statistics.
- 7 Preserves the existing destination ID during reimport.
- 8 Updates the registry.

A `.3dgmesh` contains geometry and its imported material set. A scene-assigned `.3dgmat` is an object override and is stored by the scene, Character Asset, or Prefab-not baked into the mesh during ordinary material assignment.

Skeletal import

`ImportSkeletalAssetsToContent` writes a coordinated set:

- `.3dgskmesh` for skinned vertices, skeleton copy, materials, and optionally embedded animations;
- `.3dgskel` for the reusable skeleton;
- one `.3dganim` per imported animation clip.

The mesh and animation assets depend on the skeleton ID. Animation channels also retain bone names so compatible skeletons can merge clips even when bone indices differ.

`InspectModelSource` performs a lightweight pre-import check and reports mesh, bone, and animation counts. The editor uses this to choose static or skeletal import settings.

Texture import and decoding

`ImportTextureToAsset` decodes the source into bottom-up RGBA8 pixels and stores smoothing and sRGB flags in `.3dgtex`. Reimport preserves the ID.

The image layer supports PNG-including 16-bit PNG input-JPEG, and the texture loader's supported TGA path. `TextureAssetData` is the CPU representation; `Texture` owns the uploaded OpenGL object.

Use sRGB for color/albedo textures. Use linear sampling for normal, metal/roughness, AO, height, data masks, and most lookup textures.

Runtime asset manager

`RuntimeAssetManager` owns caches for:

- `static Model`;
- `SkinnedModel`;
- `Texture`;
- `RuntimeMaterialAsset`;

- generated runtime shader programs.

Load functions return stable non-owning pointers to cached objects. Reload replaces model GPU data in place so existing ECS pointers remain valid during editor reimport.

The skeletal loader can merge additional animation sources by bone name and cache each unique model-plus-animation combination separately.

`ResolveRegistryAssets(registry)` converts unresolved ECS asset components into loaded components and returns counts plus per-asset errors.

Authored asset formats

Extension	Editor/runtime role
.3dgmatt	PBR material and optional shader parameters
.3dgshader	Typed shader graph
.particle	Particle system
.particlefx	Multi-layer particle effect
.hud	Runtime HUD
.3dgcharacter	Character configuration
.3dgprefab	Reusable object component template
.3dgclip	Reusable animation clip/action
.3dggraph	Animation state graph and blend spaces
.btgraph	Behavior tree and blackboard
.3dgaudio	Gameplay audio cue
.3dgmusic	Adaptive music states
.3dgmixer	Mixer preset
.scene	Editable or exported scene

Double-click routing in the Content browser opens each supported authored asset in its corresponding editor panel.

Import and reimport workflow

The Content browser's **Import Asset** button opens the native OS file browser. After selection, the import settings window identifies the source type, exposes the relevant options, and lets the developer choose or create the destination Content folder.

Reimport uses the recorded source path and prior import settings. Native asset IDs remain stable, preserving scene, material, graph, and prefab references.

Cooking and packaging

`AssetCooker::CookRuntimeScene` walks the runtime scene's stable dependency graph and copies only reachable assets to a relocatable output Content folder. It writes a subset registry with the same IDs. `AssetCookResult` reports the runtime scene and every cooked asset's type, path, size, and hash.

Before packaging:

- 1 Save authored assets.
- 2 Save and export the scene.
- 3 Validate the asset registry.
- 4 Resolve all missing dependencies.

- 5 Cook the exported runtime scene.
- 6 Launch the standalone player against the cooked output.

Failure modes

- **Missing dependency warning:** the referenced ID is absent from the registry. Refresh/rebuild the registry; if the file is gone, reassign the reference.
- **Asset appears after move but fails at runtime:** the authored data may still contain a legacy path without a valid ID. Re-save it after registry refresh.
- **Reimport changes references:** indicates the destination's original header was invalid or bypassed; normal reimport preserves the handle.
- **Material texture is black:** check sRGB mode, data-map channel assignment, decode errors, and whether the material references the native `.3dgtex`.
- **Imported mesh loses animation:** it was imported as static; reimport it through skeletal settings.

05 Rendering, lighting, cameras, environment, terrain, and water

Rendering layers

The engine has three related drawing paths:

1 Primitive MeshRenderer objects through the basic Renderer.

2 Imported static Model objects through DrawModel and

RenderLoadedModels.

1 ECS PBR and skeletal scenes through PbrRenderer and SkinnedRenderer.

The editor combines these paths because a scene may contain primitives, engine-owned imported meshes, skeletal characters, terrain, water, particles, and editor-only guides.

GPU resource wrappers

Type	Responsibility
Shader	Compile/link GLSL, bind program, upload uniforms, report diagnostics
VertexLayout	Attribute ordering and stride
Mesh	VAO/VBO/EBO ownership and indexed drawing
Texture	2D image upload, mipmaps, updates, and binding
Cubemap	Six-face environment texture
Framebuffer	Off-screen color/depth render target and resizing
Model	Imported/native submeshes, materials, textures, and bounds
SkinnedModel	Skeleton, weighted submeshes, and animation clips

These objects are move-only where they own OpenGL resources. Creation, destruction, and updates require a live context.

Basic renderer and models

Renderer::Draw submits a Mesh. The caller binds the shader and sets view/projection and per-object uniforms.

Model stores multiple SubMesh records and imported Phong-style materials. DrawModel binds each submesh's maps and can apply:

- a color tint;
- an albedo texture override;
- a complete ModelMaterialOverride.

Scene-assigned materials use the complete override so albedo, specular, emissive, shininess, diffuse map, and normal map replace rather than merely multiply the embedded values.

The default imported-model shader converts linear lighting to display space with tone mapping and gamma correction. Without this, valid dark material values appear nearly black.

PBR renderer

PbrRenderer draws Transform + MeshPBR entities with Cook-Torrance metallic/roughness shading. Its options control:

- ambient and fog;
- image-based lighting;
- directional, point, and spot shadows;
- shadow distance;
- extra shadow casters such as skinned models;
- screen-space and environment integrations supplied by the host.

PbrMaterial contains albedo, metallic, roughness, AO, emissive, opacity, normal/metal-rough/height maps, UV transforms, clearcoat, transmission, subsurface, sheen, anisotropy, displacement, and blend-mode settings.

The loaded-material bridge also supports a generated shader graph and per-instance parameter overrides.

Lighting

ecs::Light supports:

- directional lights using direction;
- point lights using the entity transform as position;
- spot lights using position, direction, inner/outer angles, and range;
- area lights using a source radius.

ClusteredLights implements CPU-built Forward+ light culling for OpenGL 3.3: the screen is divided into a 16×9 tile grid, with up to 128 point lights and a fixed per-tile light list. Light data is uploaded through a uniform buffer and tile indices through a texture buffer.

Shadows

System	Use
ShadowMap	Basic directional depth map
CascadedShadow	Four camera-fitted sun cascades
PointShadow	Omnidirectional cube shadow for point lights
SpotShadow	Perspective shadow for spot lights
ShadowCasterBatch	Shared static-caster submission

Cascades preserve detail close to the camera while covering the configured shadow distance. Skinned and other custom geometry must be supplied through the extra-caster callback to appear in those passes.

Shadow disappearance distance is authored in World Settings and exported as the environment shadowDistance.

Indoor darkness is produced by direct shadowing plus skylight occlusion and minimum-skylight settings. The system is dynamic; it is not a UE-style baked lightmap pipeline.

Image-based lighting

IBL builds irradiance and prefiltered environment resources used for diffuse ambient light and specular reflections. A scene can derive them from the procedural sky. Rebuild only when the environment changes enough to justify the GPU work.

Cameras

Camera is a yaw/pitch perspective camera with movement, LookAt, view and projection matrices, FOV, and near/far planes.

Related systems:

- CameraPose: serializable position, target, FOV, and clip planes.
- CameraBlend: timed transitions with easing.
- CameraSequencePlayer: ordered travel and hold shots, looping, seeking,

Catmull-Rom paths, skipping, and shot events.

- CameraShake: additive local translation, rotation, and FOV impulses.
 - CameraDirector: gameplay command/event bridge for named sequences.
 - PlayerController: first-person, third-person, isometric, and platformer
- gameplay rigs with camera collision, shoulder view, and lock-on.

The scene Camera Manager stores presets and cinematic sequences. Camera mode is selected before Play through the character or Game Mode settings; gameplay input does not toggle camera mode.

Post-processing

Framebuffer provides HDR inputs. PostProcess owns the full-screen pipeline and supports:

- tone mapping;
- bloom;
- FXAA;
- render scale;
- an ordered stack of authored post-process shader effects.

SSAO estimates contact and cavity occlusion from screen-space depth/normal data. SSR traces reflections in screen space. Their quality depends on the depth buffer, camera matrices, render resolution, and authored thresholds.

GpuProfiler measures sequential named GPU scopes using delayed query readback so profiling does not stall the current frame.

Procedural sky and world environment

DayNightCycle::At(t) returns sun/moon direction and radiance, ambient fill, day factor, sky horizon/zenith colors, and disc colors.

ProceduralSky renders that environment and supports authored clouds. World settings control time of day, skylight, sun drive/intensity, clouds, fog, and shadow behavior.

Cloud parameters include coverage, density, scale, softness, wind speed and direction, horizon height, tint, shadow enable, shadow strength, and shadow world scale. Cloud shadows modulate sun lighting independently of whether debug overlays are visible.

Terrain and grass

Heightmap stores resolution, size, origin, and height values and provides surface queries.

Terrain can:

- generate procedural fBm height;
- accept imported or edited heightmaps;
- create a render mesh;
- build an albedo texture from height, slope, and paint layers;
- answer HeightAt(x, z) for gameplay and navigation.

The editor supports landscape/heightmap import, terrain generation, layer paint, and material-derived layer colors.

`GrassField` builds GPU-instanced procedural blades from terrain paint and a style palette. Density, blade dimensions, colors, wind, layer, and maximum blade count are configurable. Grass instances are rebuilt when their terrain or style input changes.

Water

`Water` owns a configurable grid patch with procedural waves, deep/shallow colors, Fresnel reflection, sun specular, transparency, crest foam, and flow. It updates animation time, draws as a transparent forward pass, and exposes `HeightAt` for floating or contact effects.

Transparent water should render after opaque geometry and before UI. Depth write and blend state must be restored after the pass.

Culling and debug drawing

`Frustum` extracts six view planes and tests spheres or AABBs. Particle systems also have authored bounds and culling controls.

Editor visualization uses `EditorLineRenderer` for collider outlines, sockets, empty-object icons, navigation overlays, camera rails, AI vision, gizmos, and selection outlines. These guides are editor/debug presentation and do not become runtime mesh geometry.

06 Materials, textures, and shader graphs

Material asset

A `.3dgmats` stores a `RuntimeMaterialAsset`:

- stable material ID and name;
- full `PbrMaterial` surface values;
- albedo, normal, metal/rough, and height texture references;
- optional shader-graph reference;
- typed shader parameter values and texture references.

Texture and shader references use IDs plus fallback paths. Material saves record those dependencies in the asset registry.

Standard PBR channels

Channel	Meaning
Albedo	Linear/sRGB base surface color
Metallic	0 dielectric to 1 metal
Roughness	Microfacet spread; low is sharp
AO	Ambient-occlusion multiplier
Normal	Tangent-space normal perturbation
Metal/Rough	Packed data texture used for surface response
Height	Parallax/displacement input
Emissive	Light-independent output color
Emissive strength	Emission multiplier
Opacity	Transparent blend control
Clearcoat	Secondary glossy layer
Transmission	Light passing through surface
Subsurface	Diffuse scattering approximation
Sheen	Grazing fabric-like lobe
Anisotropy	Directional highlight response
Displacement	Geometry/surface offset amount

Color textures normally use sRGB. Normal, roughness, metallic, AO, height, and packed data maps must be treated as linear data.

Material Maker

The Material Maker authors `.3dgmats` documents and provides:

- isolated lit preview;
- environment rotation, light, and background controls;
- standard and advanced PBR fields;
- native engine-texture selection;
- texture-path paste/drop support;

- normal, packed metal/rough, and height maps;
- ORM packing from separate grayscale sources;
- presets, reset, load, save, and generated C++ snippets;
- optional shader assignment and parameter editing.

Saving the material writes the asset. Applying it in the Inspector writes the material reference to that object's scene record. To preserve the mesh and material as one reusable configuration, capture the object as a Prefab or Character Asset.

Material instances

Scene objects may override parameters from the assigned shader-backed material. Overrides live on the object, not in the shared material document. They are serialized with the scene and uploaded by `UploadLoadedMaterialShaderParameters`.

This allows multiple objects to share one material and vary color, scalar, vector, boolean, or texture parameters without duplicating the material.

Shader assets

A `.3dgshader` contains a typed `ShaderAsset`:

- domain: Surface, Post Process, Particle, or Unlit;
- blend mode;
- graph nodes, pins, and links;
- named typed parameters;
- graph-local IDs separate from the stable project asset ID.

`ValidateShaderAsset` checks required outputs, pin compatibility, links, and graph integrity.

`GenerateShaderSource` walks only nodes reachable from the active domain output, detects cycles, emits deterministic GLSL, maps fragment lines back to node IDs, and can generate a skinned variant.

Shader domains

Domain	Intended output
Surface	Lit material surface
Unlit	Color independent of scene lighting
Particle	Particle-specific render inputs
Post Process	Full-screen effect sampling the previous scene color

Choose the domain before building the graph because output pins and generated pipeline assumptions differ.

Shader Editor

The Shader Editor provides:

- live isolated preview;
- typed node graph and compatible-node prompt when dragging from a pin;
- right-click node actions;
- undo, redo, delete, duplicate, copy, paste, framing, alignment, and layout;
- parameters and texture selection from engine-owned assets;

- generated vertex/fragment source and node-linked diagnostics;
- automatic compile, manual compile, apply, last-valid restore, and fallback;
- surface, unlit, particle, and post-process previews.

Object Color is preview input, not an authored constant. Use a Color/Vector parameter or constant-color node when the shader needs a saved color value.

Runtime shader manager

RuntimeShaderManager caches programs by asset path and variant. A compile or reload stores:

- the program;
- diagnostics;
- asset/source hashes;
- dependency hashes;
- whether a fallback is active.

If hot reload fails after a valid compile, the previous valid program remains active. A first-time failure receives a visible domain fallback rather than a null program. DependenciesChanged detects included or referenced file changes.

Texture packing

The Material Maker can pack an ORM texture:

- red = ambient occlusion;
- green = roughness;
- blue = metallic.

The shader and material loader must agree on this channel convention. A roughness texture should not be pasted into the Normal Map slot; doing so produces invalid lighting even when decoding succeeds.

Imported-model material override

Native .3dgmesh files may contain embedded imported materials and textures. A scene-assigned .3dgmat supplies a complete override to all imported submeshes in the current implementation. It is not permanently written back into the .3dgmesh.

The display path tone-maps and gamma-corrects the linear result. Very low albedo values will still look dark by design but no longer collapse to black solely because display conversion was missing.

Troubleshooting

- **Material does not change:** confirm the object stores the .3dgmat path/ID, the runtime cache resolved it, and the correct rendering path receives the complete override.
- **Black material:** inspect albedo magnitude, light/skylight, normal-map validity, sRGB flags, shader diagnostics, and texture decode errors.
- **Could not decode image:** import the source as .3dgtex; verify supported PNG/JPEG/TGA encoding and whether it is a valid image rather than a packed or mislabeled file.
- **Missing dependency:** refresh the Content browser/registry and re-save the material after assigning native texture assets.
- **Shader compile failure:** select the diagnostic to locate its graph node,

repair the typed link or missing required output, and retain the last valid program until the new compile succeeds.

07 Animation, graphs, blend spaces, actions, sockets, and characters

Runtime data model

`Skeleton` is a topologically ordered bone list with inverse-bind offsets, local bind transforms, and the model's global inverse transform.

`Animation` contains a duration, tick rate, and a `BoneChannel` per skeleton bone. Each channel can contain position, quaternion rotation, and scale keys.

`SkinnedModel` owns weighted mesh data, a skeleton, animation clips, imported materials/textures, and bounds.

`AnimatedModel` is the per-entity ECS component. It stores:

- a shared `SkinnedModel` pointer;
- an `AnimationController`;
- the current skinning pose;
- material overrides and render-only orientation offset;
- a one-shot action layer and animation events;
- socketed static-model attachments and named gameplay sockets.

Pose evaluation

`Animator` is stateless and provides:

- bind pose;
- single-clip pose;
- cross-faded pose;
- local-pose sampling;
- local-pose blending;
- masked override layers;
- hierarchy composition into skinning matrices;
- root-translation sampling;
- bone-mask generation from a root bone and descendants.

`UpdateAnimations(registry, dt)` advances every `AnimatedModel`, evaluates its controller, blends the previous/current state, evaluates blend-space samples, layers an action if active, fires events, and rebuilds the pose.

Animation controller

`AnimationController` is a model-independent state machine. A state defines:

- name, clip, loop, and speed;
- duration;
- optional legacy activation range;
- root-motion flag;
- optional blend clip;
- 1D or 2D blend-space samples;

- blend parameters and synchronization.

Parameters are Float, Bool, or Trigger. Transitions support:

- from/to state, including any-state transitions;
- greater/equal, less, equal, and not-equal comparisons;
- multiple conditions with all/any evaluation;
- fade duration;
- normalized exit time;
- priority;
- interruption during an existing blend.

TransitionDebug() reports why each transition was or was not eligible.

Crossfades use smoothstep interpolation. Blend-space parameters are exponentially smoothed independently from raw transition parameters, keeping logic responsive while visual pose changes remain gradual.

Blend spaces

A state may have multiple (value, valueY, clip) samples:

- 1D blend spaces interpolate neighboring samples on one parameter, commonly

Speed.

- 2D blend spaces weight samples across two parameters, commonly forward and

strafe speed.

- Synchronization maps samples to a common normalized phase so walk/run cycles

do not foot-pop while blending.

Locomotion should publish raw movement state into graph parameters. The behavior tree does not choose animation clips; it drives movement intent and the animation graph reads locomotion flags such as speed, grounded, and falling.

Animation graph assets

A .3dgggraph stores reusable clips, states, parameters, transitions, blend spaces, action profiles, and events. Clips come from .3dgc clip assets and use unique graph-facing aliases.

AnimationGraphDesc is the engine-side canonical runtime description. BuildAnimationController resolves clip names with fallback indices and maps the description into a controller. The editor adapter only copies authoring fields into this descriptor.

Double-clicking a .3dgggraph opens the Graph Editor. The editor previews the graph on its configured preview model rather than whichever scene object is selected.

Clip and action assets

A .3dgc clip identifies one named animation inside a native .3dgskmesh or .3dganim source. It stores:

- display name and source clip;
- strip-root-motion, loop, and speed;
- whether it is a standalone action;
- mask root bone;
- fade-in and fade-out;
- timestamped gameplay events.

Action clips are authored in the Clip Editor but invoked from gameplay scripts. They are not automatically selected by the state graph.

Full-body action:

```

GPP
if (!Anim().IsActionPlaying()) {
    Anim().PlayActionClip("StaffAttack");
}

```

A full-body action has an empty mask and `AnimatedModel::BlocksMovement()` returns true until it completes. This provides montage-like behavior: normal locomotion cannot move the character during the action.

A masked action affects a bone subtree, such as the upper body, and does not block movement:

```

GPP
Anim().PlayMaskedAction("Cast", "spine_01", 0.08f, 0.15f);

```

Check `IsActionPlaying()` or use gameplay cooldown state to prevent input spam.

Animation events

Events are timestamped names authored on clips or profiles. As playback crosses an event, the runtime dispatches it through `AnimatedModel::onEvent` and the script input event list.

Scripts query:

```

GPP
if (WasAnimationEvent("SpawnFireball")) {
    // Spawn from a named socket here.
}

```

Events can also drive audio actions through `RuntimeAudioSystem`.

Root motion and model offsets

Strip root motion for in-place locomotion clips. Enable root motion only for states whose authored translation should intentionally drive the character.

Model offsets are render-only:

```

TEXT
world Transform
* model position offset
* model orientation offset
* model scale offset
* recenter around imported bounds

```

They correct Z-up/down-facing or backwards imports without rotating the entity's collider, movement axes, navigation, or scene gizmo.

Sockets and attachments

A character socket stores a name, bone, and local position/rotation/scale.

`AnimatedModel::SocketWorldTransform` resolves it every frame from the current pose and character transform.

Attachments mount a static model and optional material on a named socket. Gameplay can use sockets without visible attachments—for example `StaffTip` as a projectile spawn point.

The Character Editor lets the developer select sockets/attachments and edit their socket transform with the preview gizmo.

Character Asset

A `.3dgcharacter` bundles:

- skeletal mesh and material;

- animation graph and standalone action clips;
- model render offsets;
- collider;
- player-controller and camera settings;
- health and ragdoll;
- AI/nav settings, team, behavior tree, and perception;
- multiple scripts;
- sockets and attachments.

The Character Editor owns the preview model, animation graph preview, collider guide, socket gizmo, material and asset pickers, camera configuration, AI settings, and script list.

Character-editor model transforms are asset defaults and render offsets. Once placed in a scene, the scene's world transform is edited independently.

Recommended update order

- 1 Update player or AI movement and publish graph parameters.
- 2 Run gameplay scripts that may start actions or set triggers.
- 3 Call `UpdateAnimations`.
- 4 Process animation events.
- 5 Apply ragdoll pose override after physics if active.
- 6 Draw with `SkinnedRenderer`.
- 7 Draw socketed static attachments.

Troubleshooting

- **Clip name error:** use the graph alias or exact imported clip name and refresh saved clip choices after import.
- **Missing blend space:** add at least two samples and choose the controlling parameter; for 2D select the Y parameter too.
- **Snappy locomotion:** increase crossfade and blend-space smoothing; ensure sample thresholds are ordered and synchronized.
- **Model faces down/backward:** adjust render-only orientation in Character Editor, not the scene entity rotation.
- **Attack does not play:** verify the action asset is listed on the Character Asset, the script name matches it, scripts are compiled/enabled, and spam prevention is not permanently latched.
- **Socket spawn direction reversed:** derive forward from the socket world matrix or corrected character facing, not the raw imported model axis.

08 Physics, collision, controllers, joints, and ragdolls

Physics world

`PhysicsWorld::Step(registry, dt)` gathers entities with `Transform + Collider` and optional `RigidBody`, integrates motion, detects contacts, solves constraints, updates sleeping, and emits collision events.

Call it from the fixed-step loop. The main settings are gravity, solver iterations, broad-phase enable/cell size, restitution threshold, and sleeping thresholds.

Rigid bodies

`ecs::RigidBody` contains:

- linear velocity and accumulated force;
- inverse mass (0 means immovable);
- gravity toggle;
- kinematic mode;
- linear and angular damping;
- sleep state;
- CCD;
- angular velocity, torque, inverse inertia, and freeze rotation.

Use:

- no rigid body for a static collider;
- `RigidBody::Static()` for an explicitly immovable body;
- `RigidBody::Dynamic(mass)` for simulated props;
- kinematic for doors, lifts, and moving platforms driven by authored velocity

or scripts.

CCD sweeps fast bodies as a sphere and should be enabled on projectiles or other small fast objects that could tunnel through thin geometry.

Collider shapes

Supported shapes:

- Sphere
- Plane
- Box
- Capsule
- Cylinder
- Cone
- Pyramid
- Torus
- Staircase

Each collider stores restitution, friction, trigger state, object channel, and response mask. Shape-specific fields include radius, half extents, plane, capsule/cylinder half height, torus radii, and staircase count.

Cylinder is a flat-ended cylinder and is distinct from capsule. Capsule total height includes its two hemispherical ends.

Collision channels

Named object channels are bits:

- Default
- WorldStatic
- WorldDynamic
- Player
- Enemy
- Collectible
- Projectile
- CameraBlocker
- Trigger

Two colliders interact only if both masks accept the other object's channel:

```
TYPE  
(A.mask & B.layer) != 0  
and  
(B.mask & A.layer) != 0
```

The built-in CharacterBlockers mask ignores Collectible and Trigger objects. The CameraBlockers mask includes only world and camera-blocking channels. This allows coins to overlap without stopping the player and prevents camera spring arms from clipping against collectibles.

isTrigger keeps overlap detection and Enter/Stay/Exit events but disables physical response.

Solver

The solver uses:

- a uniform spatial-hash broad phase;
- shape-pair narrow-phase contact generation;
- contact manifolds with up to four points;
- sequential normal and two-axis friction impulses;
- warm-started cached impulses;
- positional correction;
- restitution slop;
- dynamic-body sleeping.

Contact caches improve stable stacks. Increasing solver iterations improves constraint quality but increases fixed-step CPU cost.

Collision events

CollisionEvent reports:

- entities A and B;
- Enter, Stay, or Exit;
- trigger status;
- contact point and normal;

- total normal impulse for solid contacts.

Events become available after Step. Gameplay uses them for triggers, audio, particle actions, hit logic, and scripts.

Do not apply projectile damage merely because the enemy fired. Damage is applied only after the projectile's swept collision hits a living target.

Queries

PhysicsWorld supports:

- raycast;
- sphere cast;
- sphere overlap;
- radial impulse.

Queries accept collision masks. Camera collision should use `CollisionLayer::CameraBlockers`; perception rays should ignore the observer and allow the intended target; projectiles should ignore their owner.

Joints

Joint supports:

- Distance constraint;
- Rope behavior on distance joints;
- Spring with stiffness and damping;
- Ball joint;
- Hinge with local axes.

Joints may connect two entities or one entity to a fixed world anchor. Authored joints are stored in the scene and rebuilt when Play begins.

Character controller

CharacterController is a kinematic upright capsule. It does not participate as a normal dynamic body. Move:

- applies gravity and jump velocity;
- sweeps and slides against blockers;
- depenetrates overlaps;
- walks shallow slopes;
- steps over ledges within `stepHeight`;
- maintains grounded state and ground normal.

PlayerController wraps it with camera-relative intent, facing, and camera rigs. Physical stair stepping can be immediate while visual/camera position is smoothed using `stairSmoothingSpeed`.

AI movement and collision

Navigation supplies horizontal desired motion. MoveAiAgent combines it with wall collision, gravity, floor probes, slope limits, and step-up behavior. Movement modes are Grounded, Falling, and Flying.

MoveAgentWithCollision performs swept horizontal motion and wall sliding. This prevents nav agents from passing through scene walls even when the navigation path itself is valid.

The collision capsule and render-only model offset must remain separate. A down-facing imported mesh should be visually corrected without rotating the AI's capsule or navigation frame.

Ragdolls

Ragdoll stores authored activation settings and transient physics-body mapping. The system creates a limited set of physics bodies for important bones, links them, disables conflicting root collision response, and can apply a death impulse.

Update order:

- 1 Apply damage.
- 2 UpdateHealth sets justDied.
- 3 ActivateRagdolls creates bodies before physics.
- 4 PhysicsWorld::Step.
- 5 UpdateRagdollPoses converts body transforms into skinning matrices.

Ragdoll runtime vectors are intentionally not serialized. They are rebuilt each Play session.

Editor physics tools

The Inspector edits collider shape, dimensions, material, trigger status, collision preset, object channel, and per-channel responses. It also exposes rigid-body, dynamic/static presets, joints, and matching collider size to the object.

Physics Status and debug controls can show collider outlines, contact events, joints, and solver statistics. Debug lines have an explicit Play-mode toggle and should not be treated as game geometry.

Troubleshooting

- **Enemy passes through walls:** confirm it has a collider, the Enemy/World masks block each other, and AI movement uses the collision-aware move path.
- **Character clips into ground:** check capsule center versus half-height, model render offset, floor channel, ground probe, and starting penetration.
- **Stair snapping:** tune step height and visual stair smoothing; ensure the staircase collider dimensions match visible steps.
- **Coin blocks player:** make it a trigger/Collectible and use a response mask that does not block Player.
- **Camera clips on pickups:** remove Collectible from camera blockers.
- **Fast projectile misses:** enable CCD or use the gameplay projectile sweep.

09 AI, navigation, perception, behavior trees, and blackboards

Navigation sources

The engine supports two navigation representations.

NavGrid is a regular XZ grid of walkable cells. AStar finds 8-directional paths, prevents diagonal corner cutting, and string-pulls the result to remove unnecessary cell-by-cell turns.

NavMesh is a list of convex polygons with shared-edge portals. It locates or snaps endpoints to polygons, searches the polygon graph, and applies the funnel algorithm to produce taut corner waypoints.

NavMeshBuilder builds a lightweight Recast-style navmesh:

- 1 Rasterize configured bounds into cells.
- 2 Mark obstacle footprints.
- 3 Expand blocked cells by agent radius.
- 4 Merge walkable cells into rectangles.
- 5 Emit convex polygons and adjacency links.

Nav Mesh Bounds Volume

The editor Nav Mesh Bounds Volume defines the world region included in the nav build. It is editor-only and excluded from runtime rendering and collision.

The editor debug overlay shows walkable and blocked areas. Red cells/segments represent non-walkable space or debug visualization; they are not physical scene geometry.

The volume must cover both agent and destination and intersect the intended walkable floor. Walls/obstacles must have colliders when the nav build derives blockage from collision geometry.

Steering

Agent stores position, velocity, speed, force, and wander state.

Available steering behaviors:

- Seek and Flee;
- Arrive;
- Pursue and Evade;
- Wander;
- obstacle avoidance;
- waypoint following.

Integrate clamps acceleration and velocity. For ECS agents, apply navigation motion through the collision-aware AI movement system rather than copying the steering position directly through walls.

Ready-made AI agent

AiAgent combines steering, path planning, perception input, and a simple Patrol/Chase/Search state machine. The host supplies target position and whether the target is currently visible. The same brain can use a NavGrid or NavMesh.

It remembers the last known target, investigates heard positions, repaths on a timer, charges directly at close visible targets, and resumes patrol after its search dwell expires.

Use this for simple agents. Use a Behavior Graph when designers need explicit tasks, decorators, services, blackboard data, or script extensions.

Perception

Vision uses:

- VisionCone range and half-angle;
- InvisionCone for pure range/angle testing;
- CanSee for cone plus physics line-of-sight.

The forward vector must come from the corrected entity/controller facing, not the raw imported mesh's visual forward axis. Otherwise the vision guide and perception face backward.

Hearing uses SoundStimulus and SoundField. Noises have position, radius, loudness, and lifetime. Agents query the loudest currently audible stimulus. Hearing currently ignores occlusion.

Ground movement

AiMovementComponent owns Grounded/Falling/Flying state, gravity, maximum fall speed, ground probe, step height, maximum slope, vertical velocity, and ground normal.

Navigation remains horizontal. MoveAiAgent adds floor contact, gravity, small-step handling, slope limits, and collision sliding. It also supports a terrain ground-height input.

UpdateAiAnimationParameters publishes engine-owned movement state to the animation graph. Behavior trees drive intent; animation graphs choose locomotion poses.

Behavior Graph data

A .btgraph contains:

- a stable asset ID;
- an indexed node list and root;
- child ordering;
- node display names and canvas positions;
- decorators and services attached to nodes;
- a typed blackboard schema and initial values.

The runtime converts it into BehaviorTree<AgentContext>. AgentContext contains steering state, target/perception input, nav sources, ECS registry/self/target, blackboard, path scratch, facing/focus outputs, and debug node statuses.

Node library

Composites:

- Sequence
- Selector
- Parallel All
- Parallel One

Decorators:

- Inverter
- Succeeder

- Failer
- Repeat
- Retry
- Cooldown
- Time Limit
- Random Chance
- script decorator

Conditions and blackboard:

- Sees Target
- Target Within
- Heard Noise
- Health Below
- Target Dead
- set/check bool and float
- float below

Actions:

- Chase
- Patrol
- Move To Target
- Wait
- Idle
- Flee
- Wander
- Attack
- Focus Target
- Clear Focus
- Investigate
- Subtree
- script task

Services include repathing and script services.

New serialized node types must be appended to `BtNodeType`; reordering existing values would break saved graphs.

Blackboard

The Blackboard stores named Bool, Int, Float, Vec3, String, and Entity values. Authored `BlackboardEntry` defaults seed the live store when an agent is built.

Use explicit keys for perception and decision state. Do not let missing health silently mean "target dead." `TargetDead` should succeed only when a valid target has a Health component whose alive state is false.

BT scripts

Subclass `ai::BtScript` to implement custom roles:

- `OnEnter`;
- Tick for tasks/services;
- Check for decorators;
- `OnExit`.

Register factories in `BtScriptRegistry`. The Behavior Graph editor lists registered classes and integrates creation templates for tasks, decorators, and services.

Script nodes receive `AgentContext`, including the ECS registry, self, target, blackboard, steering, and navigation data.

Focus and attack

`FocusTarget` enables persistent target facing while the relevant branch is active. `ClearFocus` releases it when visibility or branch state changes. Facing should update every tick so the enemy does not continue in an old direction before eventually turning.

Chase should return success or stop steering when the target is within the configured acceptance/attack radius. Attack or cast branches then run without continually pushing the agent into the player.

Projectile attacks should spawn through a script task or animation event and apply damage only on projectile collision.

Editor workflow

- Double-click `.btgraph` to open Behavior Graph.
- Select saved graphs through the panel dropdown.
- **Save** overwrites the loaded graph rather than appending extensions.
- Rename composites for readable branches.
- Drag from a node pin to empty space to open the compatible-node prompt.
- Mouse wheel zooms the graph around the cursor.
- Create BT scripts from the graph's task/decorator/service inspector.
- Assign a saved behavior tree through a dropdown on a Character Asset or AI

Agent component.

Diagnostic checklist

If an enemy stands still:

- 1 Confirm Play console says the behavior tree loaded and the agent is active.
- 2 Confirm target name or hostile auto-target/team configuration.
- 3 Confirm player and enemy teams differ and are non-neutral as intended.
- 4 Confirm the tree root and branches are valid.
- 5 Inspect live node status and blackboard.
- 6 Check `TargetDead`, visibility, and distance decorators.
- 7 Verify nav bounds cover both endpoints and a path exists.
- 8 Verify collision masks allow floor contact and block walls.
- 9 Confirm movement mode is Grounded rather than unintentionally Flying.
- 10 Confirm chase reach radius is not so large that it immediately succeeds.

10 Gameplay framework, C++ and Lua scripting, timers, and global time

Gameplay components

Health stores current/max HP, alive state, and a one-frame `justDied` pulse. Damage reduces HP; `UpdateHealth` owns the authoritative alive/death transition.

Projectile stores direction, speed, range, traveled distance, damage, sweep radius, and owner.

`UpdateProjectiles` sweeps every projectile, ignores its owner, damages only the first living Health target, destroys on any solid hit, and returns `ProjectileHit` records for VFX/audio/scoring.

Attachment drives an entity from a parent transform or parent animated bone. Character-render attachments use the richer socket system; this ECS component is useful for runtime-spawned attached entities.

Ragdoll is covered in the physics chapter.

Game mode

GameMode is the process-global game-rules layer. It tracks:

- Playing, Paused, GameOver, or Victory;
- score;
- elapsed play time;
- display message;
- optional lose-on-player-death rule.

The runtime calls `Update(registry, player, dt)` and freezes gameplay when the state is not Playing. Scripts can add score, pause/resume, win, lose, reset, or set a message.

Game Mode Settings select the player object, input enable, pause/restart permissions, initial score, lose-on-death behavior, and optional camera-mode override.

Player controller

PlayerController combines:

- kinematic capsule movement;
- walk, run, jump, gravity, slopes, and steps;
- injected `PlayerInput`;
- facing-camera or facing-movement rotation;
- smooth turn speed;
- first-person, third-person, isometric, and platformer camera modes;
- spring-arm collision and return smoothing;
- shoulder camera switching;
- target lock-on;
- stair visual/camera smoothing.

Camera mode is configured before gameplay. `PlayerInput::toggleView` remains deprecated and should not be used to change the selected game mode during Play.

The controller is input-source independent, making keyboard, gamepad, replay, or AI-generated input possible.

Camera direction

CameraDirector is the script-facing bridge to host-owned camera sequences. Scripts queue play/stop/skip commands. The editor or player resolves named sequence assets, runs CameraSequencePlayer, and sends completion, skip, and timeline events back to the director.

Scripts can query sequence state and event completion without directly owning camera resources.

Script lifecycle

Gameplay scripts subclass engine::Script:

```
class Coin : public engine::Script {
public:
    void OnCreate() override {}
    void OnUpdate(float dt) override {}
    void OnFixedUpdate(float dt) override {}
    void OnDestroy() override {}
};
```

UpdateScripts creates enabled instances, calls OnCreate once, then OnUpdate. FixedUpdateScripts calls OnFixedUpdate on created instances. ShutdownScripts calls OnDestroy and releases them.

Objects can have a primary script and multiple additional script slots. Each slot has an enabled flag, class, source path, fields, and an independent instance.

Script context

ScriptContext supplies:

- registry and self entity;
- deferred destruction queue;
- input and collision/animation events;
- audio system;
- camera shake and director;
- game mode;
- authored fields;
- deferred scene-load request.

Scripts should use the context helpers rather than retaining pointers across scene reloads.

Core Script API

Entity and world:

- Self, Registry, Transform;
- FindObject, FindTransform;
- TryGet<T>, Has<T>, Remove<T>;
- SpawnEmpty, SpawnFromObject;
- DestroySelf, Destroy;
- RequestSceneLoad.

Sockets:

- SocketTransform;
- SocketPosition.

Persistence:

- `SaveValue`, `LoadValue`;
- `SaveCheckpoint`, `LoadCheckpoint`.

Timing:

- `SetTimer`, `SetTimerByEvent`, `Delay`, `ClearTimer`;
- `BindTimerFunction`, `SetTimerByFunctionName`;
- `ClearTimerByFunctionName`, `IsTimerActive`;
- `Sequence().Do().Wait().WaitUntil()`.

Global gameplay time:

- `SetGlobalTimeDilation` and `GlobalTimeDilation`;
- `EffectiveTimeDilation`;
- `HitStop` and `IsHitStopActive`.

Normal timers consume scaled gameplay time. Hit-stop duration consumes unscaled real time so a zero-dilation freeze always releases.

Input and events:

- key/mouse down and pressed;
- mouse delta;
- trigger touching/entered/exited;
- animation-event queries.

Fields:

- `String`, `Float`, `Int`, `Bool`, `Vec3`, `Color`, `Entity`, and `Asset` lookup.

Grouped APIs

For discoverability, scripts can use:

- `Input()` for keys, mouse buttons, and deltas;
- `Camera()` for shake and cinematic sequences;
- `Particles()` for play/stop/restart/burst/clear/rate/speed/status;
- `Audio()` for source, cue, mixer, and adaptive-music controls;
- `Anim()` for actions, profiles, parameters, triggers, and movement lock.

The older flat helper methods remain available.

Spawning a projectile from a socket

Typical animation-event flow:

```

void OnUpdate(float) override {
    if (Input().MousePressed(0) && !Anim().IsActionPlaying()) {
        Anim().PlayActionClip("StaffAttack");
    }

    if (WasAnimationEvent("SpawnFireball")) {
        glm::mat4 socket;
        if (SocketTransform("StaffTip", &socket)) {
            const glm::vec3 position = glm::vec3(socket[3]);
            const glm::vec3 forward =
                glm::normalize(glm::vec3(socket[2]));
            const auto fireball =
                SpawnFromObject("FireballPrototype", position);
            if (auto* projectile = TryGet<engine::Projectile>(fireball)) {
                projectile->dir = forward;
                projectile->owner = Self();
            }
        }
    }
}

```

The exact positive/negative socket axis depends on the authored socket frame; verify it visually in Character Editor.

Script fields

ScriptField adds editable Inspector data with a type, value, optional range, tooltip, and group. Asset fields should reference Content assets rather than hard-coded external paths. Entity fields resolve by runtime object name.

Creating and compiling scripts

The editor workflow:

- 1 Create or select Content/Scripts.
- 2 Choose a template and class name.
- 3 Create the script from Inspector, Character Editor, or Behavior Graph.
- 4 Attach it from the saved-script dropdown.
- 5 Edit with the built-in editor, VS Code, Visual Studio, Rider, or a custom editor.

- 1 Compile scripts and restart.

The shared game module generates/updates registrations so the same class is available to Editor Play and the standalone player.

Hot-reload module

ScriptModule loads a shared library exporting:

```

extern "C" void RegisterScriptModule(engine::ScriptRegistry&);

```

Before unload:

- 1 ShutdownScripts.
- 2 Clear ScriptRegistry.
- 3 Unload the module.
- 4 Rebuild.
- 5 Load and register again.

Violating this sequence leaves script virtual tables pointing into unloaded code.

Error behavior

Script exceptions are caught, the failing slot is disabled, and the optional script error handler reports the problem to the editor console. Unknown class names set `missingFactory`; they do not silently instantiate a different type.

Destroy and scene-load requests are deferred until after script iteration to avoid invalidating the registry from inside a callback.

11 Audio engine, sources, cues, mixer, music, and editor

Purpose

The audio system provides playback, spatial sound, mixing, reusable sound cues, adaptive music, runtime event hooks, and editor-side waveform and mix tools.

Main runtime types

Type	Responsibility
AudioEngine	Owns the playback backend, active voices, buses, effects, snapshots, cues, and music state
AudioAsset	Describes an engine-owned audio resource and its import/playback metadata
RuntimeAudioSystem	Connects scene components and gameplay events to AudioEngine
AudioEditing	Decodes audio into editable sample buffers and writes supported output
AudioSourceComponent	Stores the authored settings for a sound-emitting scene object

The playback backend is initialized defensively. A missing output device must not prevent the rest of the engine or editor from running.

Playback models

The engine supports two useful playback lifetimes:

- **Fire-and-forget:** short effects such as impacts, UI clicks, footsteps, and one-shot spell sounds.
- **Persistent source:** a source represented by a handle that can be moved, stopped, paused, faded, or adjusted while it is playing.

For persistent world sounds, update the source transform every frame so spatial audio follows the object.

Spatial audio

Spatial sources can use:

- world position and velocity;
- minimum and maximum attenuation distance;
- rolloff;
- listener-relative direction;
- inner and outer cone angles;
- doppler response;
- obstruction or occlusion;
- per-source volume, pitch, priority, looping, and spatial toggles.

The runtime system obtains the listener from the active gameplay camera. Scene audio source positions are taken from their owning entities.

Mixing

The standard buses are:

- Master
- Music
- SFX
- Dialogue
- UI
- Ambient

Each source or cue is routed to a bus. Bus controls make it possible to change volume or mute an entire category without editing individual sounds.

Supported mix processing includes low-pass and high-pass filtering, reverb, and compression. Mix snapshots collect a group of bus and effect settings into a named state. Typical snapshots include `Default`, `Paused`, `Underwater`, `Indoor`, and `Cinematic`.

Sound cues

A cue is higher-level playback logic built from one or more sounds:

- **Random:** chooses a variation, optionally using weights.
- **Sequence:** advances through entries in order.
- **Layered:** starts multiple entries together.

Cue entries can vary volume, pitch, delay, and spatial behavior. The cue can also enforce cooldown and simultaneous-instance limits. Use cues for repeated events so variation and concurrency policy remain data-driven.

Adaptive music

Adaptive music contains named states and stems. A state controls which stems are audible and how transitions occur. BPM and beat synchronization allow a transition to line up with musical timing, while crossfades avoid abrupt cuts.

Gameplay code should request a music state, rather than manually starting and stopping every stem.

Runtime event integration

`RuntimeAudioSystem` can respond to:

- `AudioSourceComponent` creation and lifetime;
- animation events;
- collision and trigger actions;
- explicit gameplay requests;
- source/listener movement;
- periodic occlusion checks.

Animation events are the preferred way to synchronize footsteps, weapon swishes, impacts, and spell releases with an animation.

Audio editor

The editor provides:

- waveform inspection and trimming;

- cue authoring and preview;
- music state and stem setup;
- mixer buses, effects, and snapshot controls;
- playback preview;
- runtime voice and budget diagnostics.

Use the Audio Editor for source and cue authoring. Use the Audio Mixer for project-wide buses, effects, and snapshots.

Script usage

Scripts access sound through the gameplay script API. Common operations are:

- 1 play a one-shot sound or cue at a world position;
- 2 start a persistent attached source;
- 3 retain its handle;
- 4 update, fade, or stop it later;
- 5 select a mix snapshot or adaptive music state.

Use engine asset paths or registered asset identity rather than external source file paths. That keeps editor Play and exported builds consistent.

Performance guidance

- Prefer mono audio for spatial effects.
- Limit long, high-quality, simultaneously decoded sources.
- Use cue instance limits for rapidly repeated gameplay events.
- Reduce occlusion update frequency for distant sources.
- Route sounds correctly so project-wide volume controls remain cheap.
- Inspect active voices and bus levels in the audio diagnostics before changing backend settings.

Important source files

- `engine/include/engine/audio/AudioEngine.h`
- `engine/include/engine/audio/AudioAsset.h`
- `engine/include/engine/audio/AudioTypes.h`
- `engine/include/engine/audio/RuntimeAudioSystem.h`
- `engine/include/engine/audio/AudioEditing.h`
- `editor/include/EditorPanels.h`

For a task-oriented guide, also see `docs/AUDIO_SYSTEM.md`.

12 Particle simulation, modules, rendering, and scripting

Purpose

The particle system authors and runs reusable effects such as fire, smoke, sparks, dust, projectiles, trails, impacts, and magic.

Runtime architecture

The engine has two simulation paths:

- **CPU emitter:** portable and suitable for modest particle counts.
- **GPU emitter:** uses compute support when available for larger effects.

The Auto backend selects GPU simulation when the renderer and driver support it, otherwise it falls back to CPU simulation. An effect should remain valid on both paths unless it deliberately depends on a GPU-only feature.

Asset and instance

A `.particle` asset stores authored emitter data. A scene particle component references that asset. At runtime, `RuntimeParticleSystem` owns live emitter instances and updates them independently from the editor preview.

An effect may contain more than one emitter-for example:

- a bright fireball core;
- an additive trail;
- sparks;
- an impact burst.

Keep asset authoring separate from the instance transform and playback state.

Module pipeline

Modules execute in a fixed pipeline:

Spawn stage

Defines how particles begin:

- spawn rate, burst count, and burst interval;
- maximum particle count;
- lifetime range;
- spawn shape and dimensions;
- initial position;
- initial velocity, direction, and spread.

Update stage

Changes living particles:

- acceleration and other forces;
- rotation;

- color over life;
- size over life;
- collision response;
- drag and other movement behavior.

Render stage

Controls presentation:

- sprite or billboard rendering;
- texture and blend behavior;
- trails or ribbons;
- renderer-facing size and color behavior.

Required modules cannot be removed. Module order is normalized and validated before the effect is compiled into a CPU or GPU emitter pipeline.

Playback

System playback includes:

- enabled and autoplay state;
- loop behavior;
- prewarm;
- duration and start delay;
- simulation speed;
- local-space or world-space simulation.

Use local space for an effect that must remain attached to a moving object. Use world space when spawned particles should remain where they were emitted.

Editor workflow

The Particle Editor contains:

- 1 an isolated preview viewport;
- 2 presets such as fire, smoke, sparks, magic, and dust;
- 3 the ordered module stack;
- 4 per-module settings;
- 5 playback and simulation controls;
- 6 system/effect save controls.

Select a module row to expose its settings. The stage headings organize execution; the editable values belong to the selected module and system sections. Previewing in the editor does not automatically place the effect in the scene.

To use an effect in a level:

- 1 save the `.particle` asset under Content;
- 2 add a particle system component to an empty object or effect prototype;
- 3 select the asset;
- 4 enable it or start it through a script;
- 5 confirm the object and effect are inside the active camera view.

Script usage

Gameplay scripts can:

- spawn an effect asset at a transform;
- attach an effect to an entity or socket;
- start, stop, pause, or restart an instance;
- set runtime parameters;
- destroy transient effects after completion.

For projectiles, spawn the projectile object and its trail separately from the impact effect. Apply damage only after the projectile collision callback, then spawn the impact effect at the confirmed contact position.

Sockets and local direction

When spawning from a character:

- 1 resolve the named socket from the animated model;
- 2 obtain its world transform after animation has updated;
- 3 derive forward direction from that transform;
- 4 offset the initial position slightly beyond the weapon;
- 5 ignore collision with the owner for the initial projectile.

This prevents effects from appearing at the character origin or traveling backward because a model-space axis was treated as world-space forward.

Diagnostics

If an effect does not appear:

- verify the saved asset path resolves through project content;
- verify a runtime component or explicit spawn call exists;
- check enabled, autoplay, delay, duration, and loop;
- inspect lifetime, size, alpha, texture, and maximum particle count;
- confirm the camera and spawn transform;
- confirm required modules passed validation;
- switch from Auto to CPU temporarily to isolate GPU capability problems;
- inspect the console for asset or shader loading failures.

Important source files

- engine/include/engine/graphics/ParticleSystem.h
- engine/include/engine/graphics/GpuParticleSystem.h
- engine/include/engine/graphics/ParticleRenderer.h
- engine/include/engine/graphics/RuntimeParticleSystem.h
- engine/include/engine/assets/ParticleAsset.h
- editor/include/ParticleEditorPanel.h
- editor/include/ParticlePresets.h
- editor/include/ParticleAsset.h

13 Runtime UI, HUD documents, bindings, fonts, and editor

Purpose

The UI layer covers immediate editor UI, runtime game HUD assets, text rendering, bound gameplay values, buttons, menus, and world-space health bars.

UI layers

Layer	Use
ImGuiLayer	Editor panels, docking, controls, debug tools
UI	Lightweight engine UI drawing helpers
Hud	Authored runtime HUD assets and binding/action evaluation
TextRenderer	Bitmap and TrueType text rendering

Editor ImGui windows are not the runtime game HUD. A HUD must be saved as an asset and selected by the scene or game-mode configuration.

HUD widgets

The HUD asset supports:

- Panel
- Text
- Bar
- Button
- Image

Widgets store visibility, anchor, pixel offset, size, color, and type-specific properties. Anchors make a layout adapt to changes in the game viewport.

Data bindings

Text and bars can read named runtime values. A text pattern containing `{}` is formatted with the bound value.

Typical values include:

- `health` and `max_health`;
- `score`;
- `time`;
- `gamestate`;
- ammunition, mana, objective progress, or custom script values.

The binding key is case-sensitive. The source kind must match the value type: for example, a time value should use a float source and a game-state label should use a string source.

The runtime host builds the HUD context each frame. Scripts publish or update named values through the script API; merely typing a key into the HUD editor does not create a live gameplay value.

Actions and buttons

Buttons emit named actions. Gameplay or menu logic consumes those actions to:

- start or restart a level;
- resume or pause;
- open another menu;
- change settings;
- quit or return to a title screen.

Keep button layout in the HUD asset and game-state changes in gameplay code.

Health display

Player health bars usually bind to the active player values in the HUD context. World-space enemy health bars use the entity's health component, project the world position into screen coordinates, and hide when the entity is not visible or no longer alive.

Damage should update the health component first. HUD code reads that state; it should not own combat rules.

HUD Editor

The HUD Editor provides:

- widget creation and deletion;
- a layout preview;
- anchors, offsets, size, colors, and text;
- binding source and key;
- button actions;
- save, load, new, and Use in Scene operations.

Double-clicking a .hud asset in the Content browser opens this panel.

Main-menu workflow

- 1 Create a new HUD asset.
- 2 Add a background panel or image.
- 3 Add title text.
- 4 Add Play, Options, and Quit buttons.
- 5 assign named actions to the buttons.
- 6 Save the asset.
- 7 Select it as the menu HUD in game-mode or scene settings.
- 8 Handle the actions in gameplay code.

Gameplay HUD workflow

- 1 Add a player health bar.
- 2 Bind current and maximum values.
- 3 Add score and time text using {}.
- 4 Publish matching named values from the gameplay scripts.

- 5 Set the HUD as the gameplay HUD.
- 6 Enter Play and inspect the runtime version, not only the editor preview.

Troubleshooting stale values

- Confirm the active HUD is the edited asset.
- Confirm the runtime context publishes the exact key.
- Confirm source types match.
- Confirm the script is attached, enabled, compiled, and running.
- Confirm the value is updated during runtime rather than only initialized.
- Confirm a saved HUD was reloaded after changes.
- Check that editor preview placeholders are not being mistaken for live data.

Important source files

- `engine/include/engine/ui/Hud.h`
- `engine/include/engine/ui/UI.h`
- `engine/include/engine/ui/ImGuiLayer.h`
- `engine/include/engine/graphics/TextRenderer.h`
- `engine/include/engine/graphics/TrueType.h`
- `editor/include/HudEditorPanel.h`

14 Editor application, panels, asset editors, and Play mode

Purpose

The editor is the authoring host for projects, scenes, engine-owned assets, components, scripts, characters, AI, animation, particles, shaders, audio, cameras, and runtime preview.

Editor shell

EditorApp coordinates the window, renderer, scene, panels, project state, input, play mode, and asset tools. EditorDockspace builds the ImGui docking layout. The viewport remains the central scene view while tool panels can be docked, floated, shown, or hidden.

The main editor state is split among:

- EditorProject: project paths and project settings;
- EditorScene: editable objects and authored component data;
- EditorRuntimeController: Play/Stop lifetime and runtime preview;
- EditorContentController: content browser operations and asset opening;
- EditorAssets: content classification and editor asset helpers;
- EditorViewport: viewport bounds, focus, picking, and render presentation.

Scene interaction

The scene tools include:

- hierarchy selection and filtering;
- inspector component editing;
- transform gizmos for move, rotate, and scale;
- world/local orientation;
- snapping;
- camera navigation and framing;
- drag-and-drop asset placement;
- line-based collision, socket, navigation, and debug overlays.

Model import orientation belongs to render/model transform data. Scene translation, rotation, and scale remain the object's world transform. This separation prevents a model-axis correction from changing gizmo directions.

Project and scene workflow

Projects define a Content root. Scenes are stored beneath Content, normally in Content/Scenes. Save overwrites the active scene; Save As creates a new scene. The last successfully saved scene is recorded and restored when the project is reopened.

Play mode creates a runtime representation from the authored scene. Stopping Play restores the editor state rather than retaining transient runtime changes.

Content browser

The Assets panel supports:

- folder navigation and creation;
- rename, copy, cut, paste, delete;
- system file browsing for imports;
- selection of the destination project folder;
- import and reimport;
- display names without exposing the entire absolute path;
- asset-specific icons and classifications;
- double-click routing to the correct editor.

Engine-owned asset metadata allows imports to remain stable even if the source file is moved outside the project. Registry identity and dependencies are used by authoring tools and cooked builds.

Asset opening

Double-clicking an asset should route as follows:

Asset	Panel
scene	Scene loader
material	Material Maker
shader or shader graph	Shader Editor
particle	Particle Editor
HUD	HUD Editor
character	Character Editor
behavior tree graph	Behavior Graph
animation graph	Graph Editor
action clip	Clip Editor
prefab	Prefab tools
audio/cue	Audio Editor

Inspector

The Inspector displays only properties relevant to the selected object and its components. Rendering and animation sections must follow the selected asset type-for example, skeletal animation preview controls are hidden for static meshes and primitives.

Components are added through the Add Component workflow. Empty objects provide a transform and component host without a visible primitive.

Specialized panels

World and game

- **World Settings:** atmosphere, sky, clouds, lighting, shadows, post-processing, and debug visibility distances.
- **Game Mode Settings:** player, gameplay HUD, startup behavior, and a camera

mode chosen before Play.

- **Camera Manager:** saved cameras, shake preview, camera zones, and sequences.

Visual content

- **Material Maker:** PBR materials and engine texture selection.
- **Shader Editor:** typed node graph, compilation, parameters, textures, and domain output.
- **Particle Editor:** isolated effect preview and module stack.
- **HUD Editor:** widget layout, bindings, and actions.

Character and animation

- **Character Editor:** model, materials, collider, movement, camera, animation graph, sockets, attachments, scripts, and AI configuration.
- **Graph Editor:** animation states, blend spaces, parameters, and multi-term transitions.
- **Clip Editor:** standalone action clips, playback rules, and animation events.
- **Animation Preview:** direct clip/model inspection and diagnostics.

Character preview transforms are asset-level render corrections. When a character is placed in a scene, its scene transform is edited independently.

AI

- **Behavior Graph:** behavior-tree nodes, decorators, services, tasks, blackboard, integrated BT scripts, save/overwrite, pan, and zoom.
- **Gameplay Debug:** AI state, perception, navigation, physics, and gameplay overlays.

Audio

- **Audio Editor:** source, waveform, cue, and adaptive-music authoring.
- **Audio Mixer:** buses, effects, snapshots, and runtime monitoring.

Engineering and diagnostics

- **Console:** engine and editor log.
- **Profiler:** CPU/GPU frame costs.
- **Physics Status:** bodies, contacts, and simulation status.
- **Script API:** available script-facing operations.
- **Script Debug:** attached scripts, compile/runtime state, and errors.

Script authoring

The editor stores user-facing scripts in `Content/Scripts` while integrating them with the shared game module. Script creation:

- 1 validates a class name;
- 2 generates the selected template;

- 3 updates generated registration/includes;
- 4 exposes the script in saved-script dropdowns;
- 5 invokes the project compiler;
- 6 reports actionable errors in the Console.

Objects and character assets can attach multiple scripts. Attach operations must respect object locking and show why an attachment was rejected.

External editor choices include Visual Studio Code, Visual Studio, and Rider.

Debug drawing

Editor guides use line rendering instead of temporary cube meshes. The global and per-system debug toggles cover:

- colliders;
- physics contacts;
- navigation areas;
- AI paths and vision;
- sockets and attachments;
- camera rails and volumes.

Debug guides should not appear in gameplay unless their Play-mode toggle is enabled.

Important source files

- `editor/include/EditorApp.h`
- `editor/include/EditorProject.h`
- `editor/include/EditorScene.h`
- `editor/include/EditorRuntimeController.h`
- `editor/include/EditorContentController.h`
- `editor/include/EditorPanels.h`
- `editor/include/EditorViewport.h`
- `editor/include/EditorGizmo.h`
- `editor/include/EditorLineRenderer.h`

15 Build, standalone player, packaging, diagnostics, and tests

Build structure

The repository builds several major targets:

- engine: reusable runtime library;
- editor: authoring application;
- game: shared user-script/game module;
- player: standalone runtime application;
- test executables registered with CTest.

CMake defines target dependencies and platform libraries. Runtime features should live in engine; authoring-only behavior should live in editor. Gameplay scripts belong to the shared game module so both Editor Play and the standalone player execute the same code.

Core dependencies

The engine integrates graphics, window/input, UI, asset import, audio, and physics libraries through its CMake configuration. Optional capabilities must degrade safely-for example, GPU particles fall back to CPU and audio device failure does not prevent editor startup.

Runtime startup

The standalone player:

- 1 resolves the project and runtime content paths;
- 2 loads runtime configuration and game-mode settings;
- 3 loads the startup scene;
- 4 loads an optional `.3dgworld` manifest and its persistent level;
- 5 initializes registered C++ and Lua gameplay scripts;
- 6 constructs runtime and level-streaming systems;
- 7 enters the application update/render loop.

Editor Play follows the same runtime rules but starts from an exported clone of the currently authored scene.

Export and cooking

Runtime export converts editor-authored state into player-readable data. Cooking:

- resolves asset references;
- validates dependency closure;
- copies or transforms runtime payloads;
- removes editor-only state;
- writes registry/runtime metadata;
- reports missing assets before packaging.

The player must not depend on source files outside the project after cooking. Use engine asset identity or project-relative content paths.

Validation

Validate before export to catch:

- missing asset dependencies;
- invalid scene references;
- unresolved scripts;
- shader compile failures;
- malformed animation, behavior, character, HUD, or particle assets;
- assets that still reference an external import source as their runtime data.

Warnings about absent registry dependencies should be repaired by importing or reassigning the referenced engine asset, then saving the owner asset.

Automated tests

The current test targets cover:

Target	Area
particle_tests	particle data and simulation behavior
animation_movement_tests	animation-driven movement
camera_manager_tests	camera selection and blending
audio_system_tests	audio types, cues, and runtime behavior
shader_asset_tests	shader asset serialization/validation
material_shader_tests	material and shader integration
script_system_tests	script lifecycle and API
world_streaming_tests	world manifests, streaming policy, transforms, activation, and unload
runtime_ai_tests	AI and behavior runtime
image_decode_tests	texture decode paths, including PNG handling
editor_assets_tests	editor content classification/routing
asset_registry_tests	identity and dependency registry
asset_cooker_tests	runtime cooking
static_mesh_asset_tests	static mesh native assets
skeletal_asset_tests	skeletal native assets

Run the full suite after changing shared serialization or runtime ownership. Run the focused target while iterating on a single system.

Runtime diagnostics

The editor exposes:

- CPU and GPU profiler timings;
- engine Console logs;
- asset validation messages;
- script compile and runtime state;
- physics and AI debug overlays;
- particle and audio runtime statistics.

A frame-cost investigation should distinguish:

- scene submission/build time;
- CPU rendering;
- GPU scene rendering;
- editor UI cost;
- physics, animation, AI, particle, and audio updates.

This prevents optimizing the wrong subsystem.

Extension checklist

When adding a new engine asset or runtime system:

- 1 define its runtime data and ownership;
- 2 define serialization and versioning;
- 3 register its extension/type in asset classification;
- 4 add registry dependency extraction;
- 5 add import and reimport if it has source data;
- 6 add Content-browser double-click routing;
- 7 add validation and cooking;
- 8 expose only necessary script APIs;
- 9 add editor authoring UI;
- 10 add focused tests;
- 11 document its runtime ordering and failure behavior.

Important source files

- root CMakeLists.txt
- engine/CMakeLists.txt
- editor/CMakeLists.txt
- game/CMakeLists.txt
- player/CMakeLists.txt
- tests/CMakeLists.txt
- engine/include/engine/core/Application.h
- engine/include/engine/assets/AssetCooker.h
- engine/include/engine/scene/RuntimeSceneLoader.h
- engine/include/engine/scene/WorldManifest.h
- engine/include/engine/scene/LevelStreamingManager.h
- editor/include/RuntimeSceneExporter.h

16 Public source coverage appendix

This appendix is the coverage checklist for the systems reference. It maps all public headers currently found under engine/include/engine and editor/include to the chapter that explains the system.

Inventory at the time of writing: **109 runtime headers** and **35 editor headers**.

Runtime: AI

Chapter: AI and navigation

- AgentCollision.h
- AiAgent.h
- AiMovement.h
- AStar.h
- BehaviorGraph.h
- BehaviorTree.h
- Blackboard.h
- BtScript.h
- NavGrid.h
- NavMesh.h
- NavMeshBuilder.h
- Perception.h
- StateMachine.h
- Steering.h

Runtime: animation

Chapter: Animation

- AnimatedModel.h
- AnimationController.h
- AnimationGraphDesc.h
- Animator.h
- Skeleton.h

Runtime: assets

Chapter: Assets. Material and shader-specific types are also covered by Materials and shaders.

- AssetCooker.h
- AssetIdentity.h
- AssetReference.h
- AssetRegistry.h
- MaterialAssetLoader.h
- ParticleAsset.h
- RuntimeAssetManager.h

- [RuntimeShaderManager.h](#)
- [ShaderAsset.h](#)
- [ShaderGraphCompiler.h](#)
- [SkeletalAsset.h](#)
- [StaticMeshAsset.h](#)
- [TextureAsset.h](#)

Runtime: audio

Chapter: Audio

- [AudioAsset.h](#)
- [AudioEditing.h](#)
- [AudioEngine.h](#)
- [AudioTypes.h](#)
- [RuntimeAudioSystem.h](#)

Runtime: core

Chapter: Core architecture

- [Application.h](#)
- [Config.h](#)
- [HighPerformanceGPU.h](#)
- [Paths.h](#)
- [Window.h](#)

Runtime: ECS and scenes

Chapter: ECS and scenes

- [Components.h](#)
- [Entity.h](#)
- [Prefab.h](#)
- [Registry.h](#)
- [RuntimeSystems.h](#)
- [Systems.h](#)
- [LevelStreamingManager.h](#)
- [RuntimeSceneLoader.h](#)
- [WorldManifest.h](#)

Runtime: gameplay and scripting

Chapter: Gameplay and scripting

- [CameraDirector.h](#)
- [GameMode.h](#)
- [GameplayComponents.h](#)

- [GameplaySystems.h](#)
- [LuaScript.h](#)
- [PlayerController.h](#)
- [RagdollSystem.h](#)
- [Script.h](#)
- [ScriptModule.h](#)

Runtime: rendering foundations

Chapter: Rendering

- [Camera.h](#)
- [CameraBlend.h](#)
- [CameraSequence.h](#)
- [CameraShake.h](#)
- [Cubemap.h](#)
- [Framebuffer.h](#)
- [Frustum.h](#)
- [GpuProfiler.h](#)
- [ImageDecode.h](#)
- [Mesh.h](#)
- [Model.h](#)
- [ObjData.h](#)
- [Primitives.h](#)
- [Renderer.h](#)
- [Shader.h](#)
- [SkinnedModel.h](#)
- [SkinnedRenderer.h](#)
- [Texture.h](#)
- [VertexLayout.h](#)

Runtime: lighting, environment, and post-processing

Chapter: Rendering

- [CascadedShadow.h](#)
- [ClusteredLight.h](#)
- [DayNightCycle.h](#)
- [GrassField.h](#)
- [IBL.h](#)
- [PbrRenderer.h](#)
- [PointShadow.h](#)
- [PostProcess.h](#)
- [ProceduralSky.h](#)

- ShadowCasters.h
- ShadowMap.h
- Skybox.h
- SpotShadow.h
- SSAO.h
- SSR.h
- Terrain.h
- Water.h

Runtime: particles

Chapter: Particles

- GpuParticleSystem.h
- ParticleRenderer.h
- ParticleSystem.h
- RuntimeParticleSystem.h

Runtime: text and UI

Chapter: UI and HUD

- TextRenderer.h
- TrueType.h
- Hud.h
- ImGuiLayer.h
- UI.h

Runtime: math

Spline evaluation is used by camera and cinematic paths and is documented in Rendering.

- Spline.h

Runtime: physics

Chapter: Physics

- CharacterController.h
- PhysicsComponents.h
- PhysicsWorld.h

Editor: animation, characters, behavior, particles, shaders, and HUD

Chapters: Editor, Animation, AI, Particles, Materials and shaders, and UI/HUD.

- AnimationClipAsset.h
- AnimationGraphAsset.h
- AnimationGraphBuilder.h
- AnimationGraphEditorPanel.h

- BehaviorGraphPanel.h
- CharacterAsset.h
- CharacterEditorPanel.h
- ClipEditorPanel.h
- GameBtScripts.h
- HudEditorPanel.h
- ParticleAsset.h
- ParticleEditorPanel.h
- ParticlePresets.h
- ShaderEditorPanel.h

Editor: application, project, scene, content, and interaction

Chapter: Editor

- EditorApp.h
- EditorAssets.h
- EditorCameraController.h
- EditorContentController.h
- EditorDockspace.h
- EditorDragDrop.h
- EditorGeneratedScriptTools.h
- EditorGizmo.h
- EditorLineRenderer.h
- EditorLog.h
- EditorMouseController.h
- EditorPanels.h
- EditorProject.h
- EditorRuntimeController.h
- EditorScene.h
- EditorScriptTools.h
- EditorTransformController.h
- EditorViewport.h
- NativeDialog.h
- PrefabAsset.h
- RuntimeSceneExporter.h

Coverage rule

When a public header is added, its owning system should update the appropriate chapter and add the header to this appendix. Internal .cpp files implement these APIs; they are deliberately not duplicated here as separate "systems."